

```
In [1]: import pandas as pd
import numpy as np
import seaborn as sns      #makes visualisations
import matplotlib.pyplot as plt    #showing graphs
```

```
In [2]: df = pd.read_csv(r"C:\Users\Rashid\Downloads\Telecom_Churn.csv")
df
```

Out[2]:

	state	account length	area code	phone number	international plan	voice mail plan	number vmail messages	total day minutes	total day calls	total day charge	...
0	KS	128	415	382-4657	no	yes	25	265.1	110	45.07	...
1	OH	107	415	371-7191	no	yes	26	161.6	123	27.47	...
2	NJ	137	415	358-1921	no	no	0	243.4	114	41.38	...
3	OH	84	408	375-9999	yes	no	0	299.4	71	50.90	...
4	OK	75	415	330-6626	yes	no	0	166.7	113	28.34	...
...	...	...	...	...	...	...	...	...	...	...	...
3328	AZ	192	415	414-4276	no	yes	36	156.2	77	26.55	...
3329	WV	68	415	370-3271	no	no	0	231.1	57	39.29	...
3330	RI	28	510	328-8230	no	no	0	180.8	109	30.74	...
3331	CT	184	510	364-6381	yes	no	0	213.8	105	36.35	...
3332	TN	74	415	400-4344	no	yes	25	234.4	113	39.85	...

3333 rows × 21 columns

```
In [3]: df.head()
```

Out[3]:

	state	account length	area code	phone number	international plan	voice mail plan	number vmail messages	total day minutes	total day calls	total day charge	...	total eve call
0	KS	128	415	382-4657	no	yes	25	265.1	110	45.07	...	99
1	OH	107	415	371-7191	no	yes	26	161.6	123	27.47	...	100
2	NJ	137	415	358-1921	no	no	0	243.4	114	41.38	...	110
3	OH	84	408	375-9999	yes	no	0	299.4	71	50.90	...	80
4	OK	75	415	330-6626	yes	no	0	166.7	113	28.34	...	120

5 rows × 21 columns

In [4]: `df.isna().sum()`

```
Out[4]: state                0
account length              0
area code                  0
phone number               0
international plan         0
voice mail plan            0
number vmail messages      0
total day minutes          0
total day calls            0
total day charge           0
total eve minutes          0
total eve calls            0
total eve charge           0
total night minutes        0
total night calls          0
total night charge         0
total intl minutes         0
total intl calls           0
total intl charge          0
customer service calls     0
churn                     0
dtype: int64
```

In [5]: `df.shape`

Out[5]: (3333, 21)

In [6]: `df.describe()`

Out[6]:

	account length	area code	number vmail messages	total day minutes	total day calls	total day charge	total eve minutes
count	3333.000000	3333.000000	3333.000000	3333.000000	3333.000000	3333.000000	3333.000000
mean	101.064806	437.182418	8.099010	179.775098	100.435644	30.562307	200.980348
std	39.822106	42.371290	13.688365	54.467389	20.069084	9.259435	50.713844
min	1.000000	408.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	74.000000	408.000000	0.000000	143.700000	87.000000	24.430000	166.600000
50%	101.000000	415.000000	0.000000	179.400000	101.000000	30.500000	201.400000
75%	127.000000	510.000000	20.000000	216.400000	114.000000	36.790000	235.300000
max	243.000000	510.000000	51.000000	350.800000	165.000000	59.640000	363.700000

In [7]: df.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3333 entries, 0 to 3332
Data columns (total 21 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   state                                3333 non-null   object
1   account length                       3333 non-null   int64
2   area code                           3333 non-null   int64
3   phone number                        3333 non-null   object
4   international plan                  3333 non-null   object
5   voice mail plan                     3333 non-null   object
6   number vmail messages               3333 non-null   int64
7   total day minutes                   3333 non-null   float64
8   total day calls                     3333 non-null   int64
9   total day charge                    3333 non-null   float64
10  total eve minutes                   3333 non-null   float64
11  total eve calls                     3333 non-null   int64
12  total eve charge                    3333 non-null   float64
13  total night minutes                 3333 non-null   float64
14  total night calls                   3333 non-null   int64
15  total night charge                  3333 non-null   float64
16  total intl minutes                  3333 non-null   float64
17  total intl calls                    3333 non-null   int64
18  total intl charge                   3333 non-null   float64
19  customer service calls              3333 non-null   int64
20  churn                              3333 non-null   bool
dtypes: bool(1), float64(8), int64(8), object(4)
memory usage: 524.2+ KB

```

In [11]: *#Data visualisation with seaborn:*

```

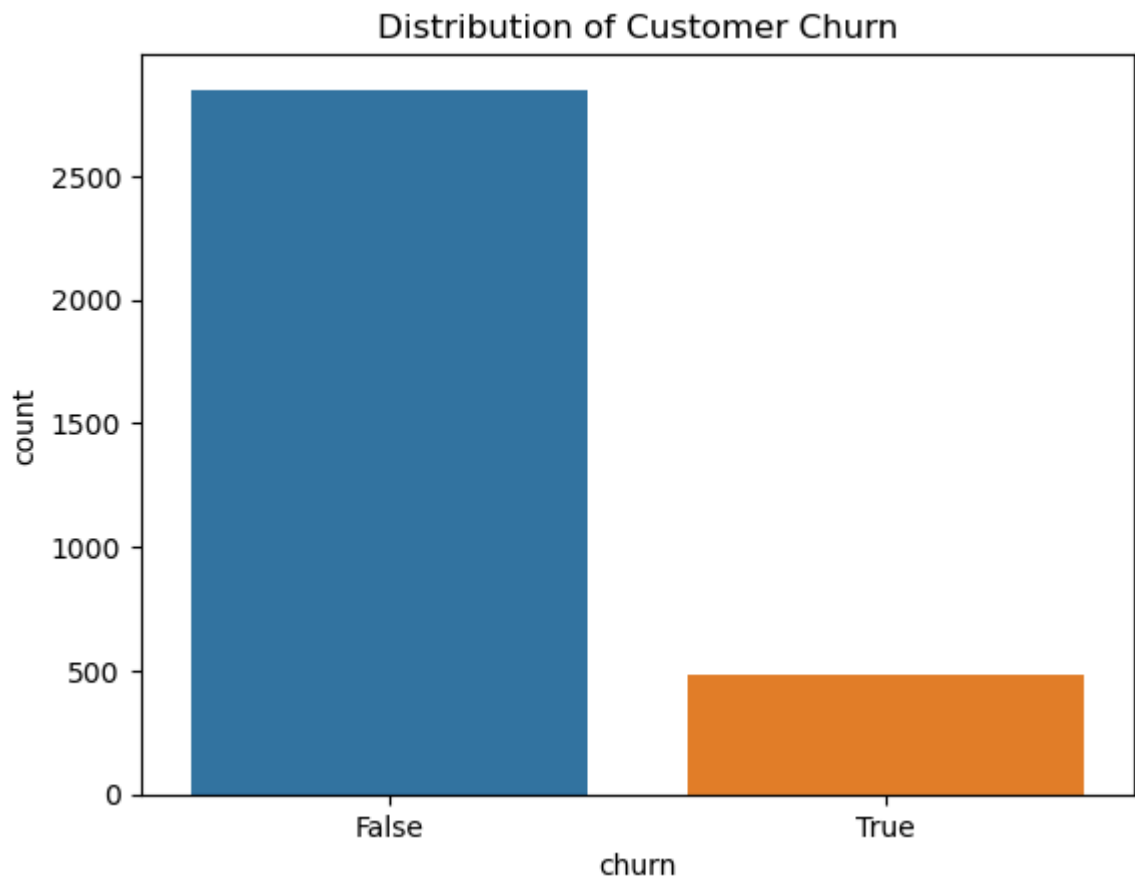
# Visualise the distribution of customer churn

sns.countplot(x="churn", data=df)      #sns.countplot() counts how many customers are

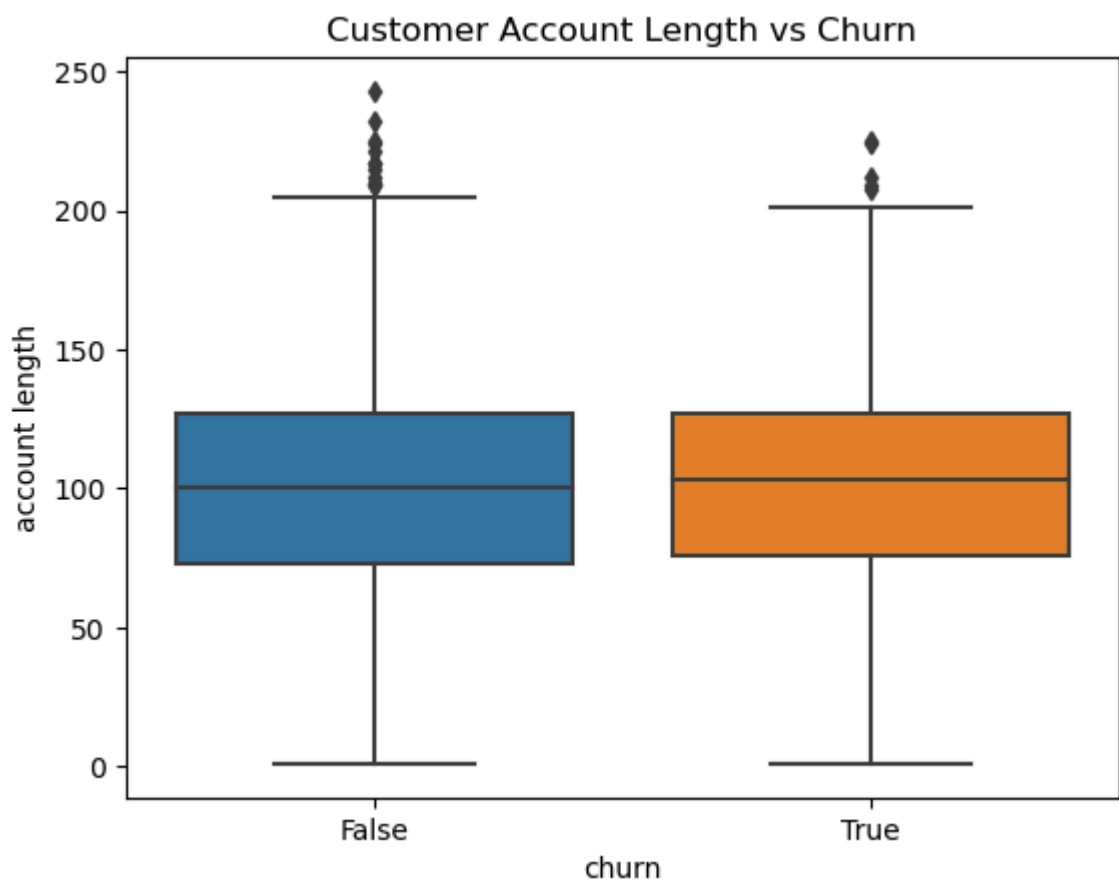
plt.title("Distribution of Customer Churn")
plt.show()

#False -> customers who did not leave
#True -> customers who did leave

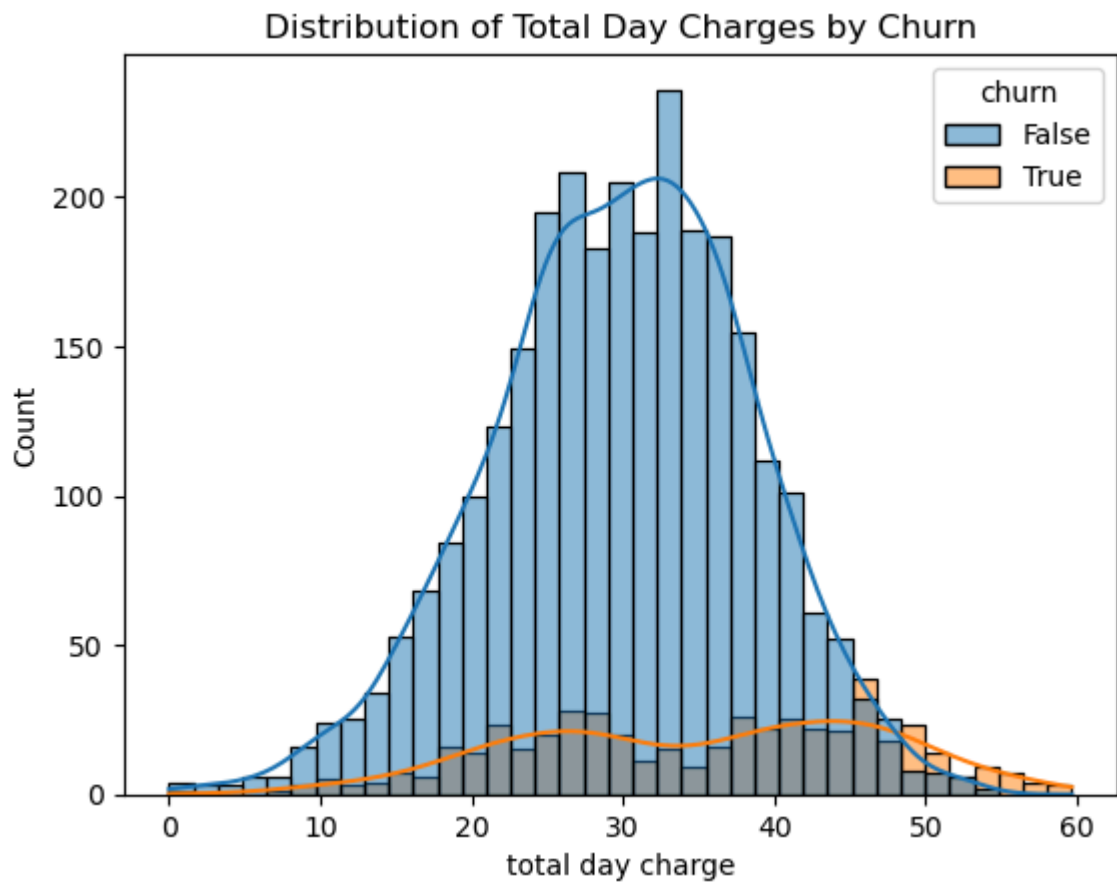
```



```
In [12]: # Visualise the relationship between customer account length and churn  
  
sns.boxplot(x="churn", y="account length", data=df)      #account length= how long  
  
plt.title("Customer Account Length vs Churn")  
plt.show()
```

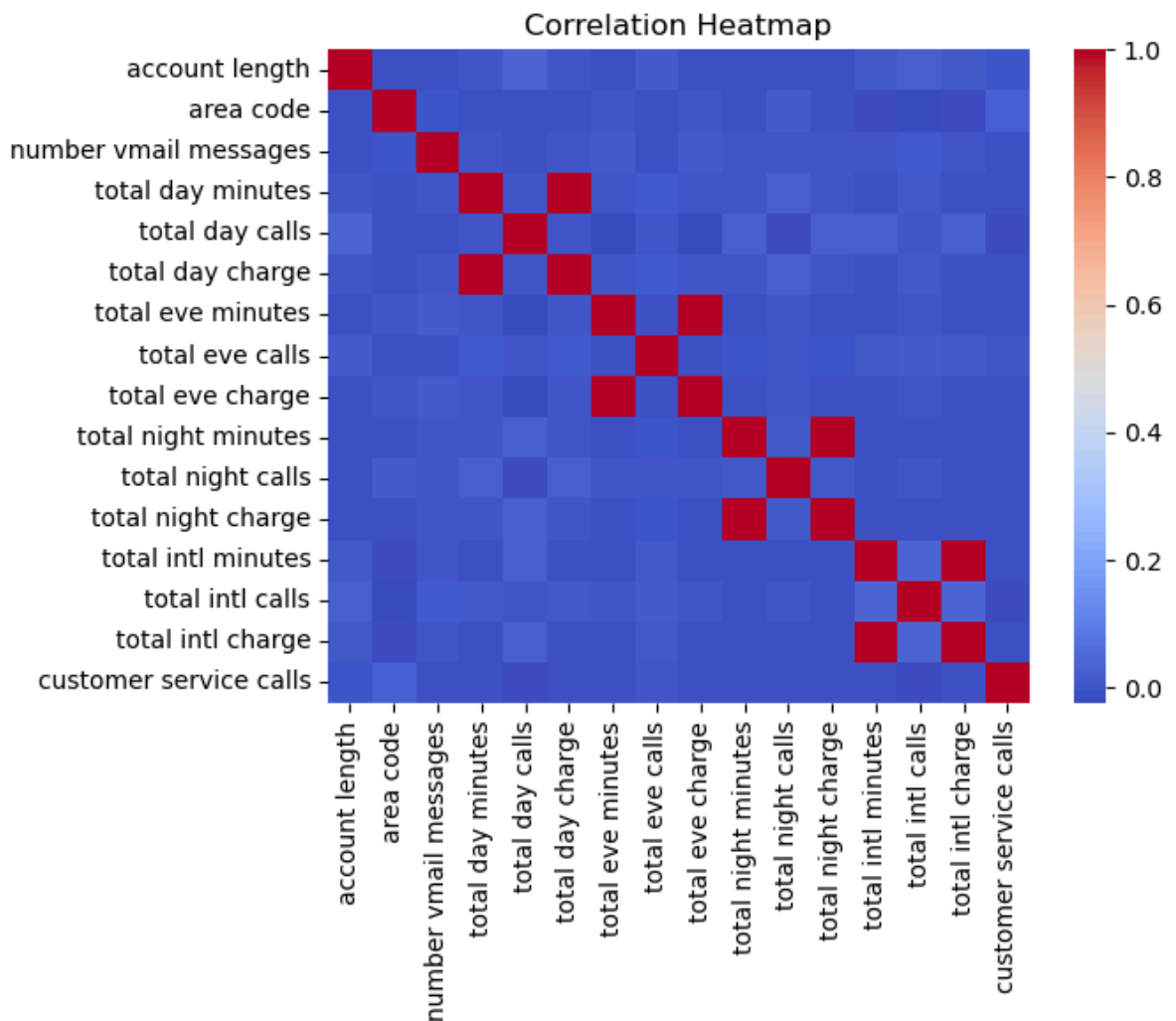


```
In [13]: # Visualise the distribution of total day charges for churned and non-churned custo
sns.histplot(x="total day charge", hue="churn", data=df, kde=True) #→ puts custome
plt.title("Distribution of Total Day Charges by Churn") #separates customers who c
plt.show()
```



```
In [14]: #Correlation Heatmap:
numeric_df = df.select_dtypes(include="number") #Take only the columns containi
corr = numeric_df.corr() #how strongly each numerical column is related to ev
sns.heatmap(corr, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

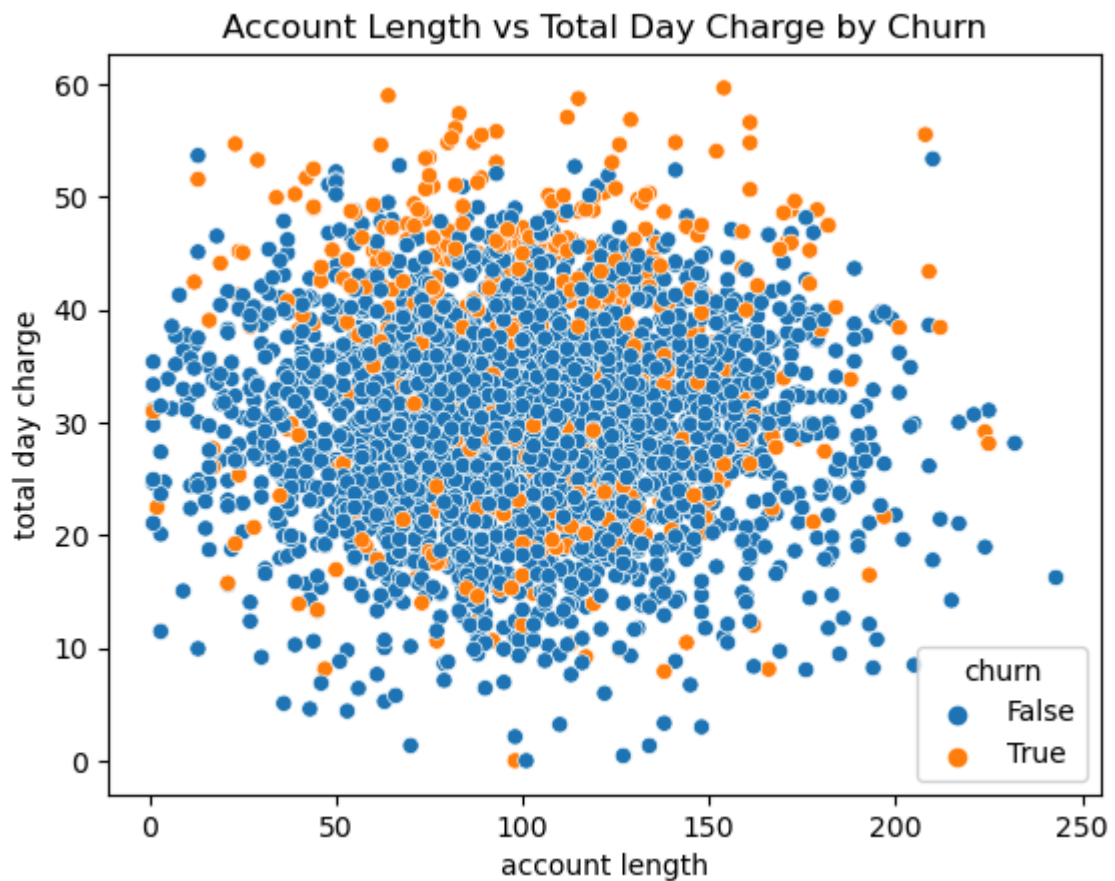
#+1 → strong positive relationship
#0 → little/no linear relationship
#-1 → strong negative relationship
```



In [15]: *# Visualise the relationship between account length and total day charges*

```
sns.scatterplot(
    x="account length",
    y="total day charge",
    hue="churn",
    data=df
)

plt.title("Account Length vs Total Day Charge by Churn")
plt.show()
```



```
In [16]: # Compare average numerical values for customers who stayed and customers who churned
churn_comparison = df.groupby("churn").mean(numeric_only=True)
churn_comparison
```

```
Out[16]:
```

	account length	area code	number vmail messages	total day minutes	total day calls	total day charge	total eve minutes	total eve calls
churn								
False	100.793684	437.074737	8.604561	175.175754	100.283158	29.780421	199.043298	100.03859
True	102.664596	437.817805	5.115942	206.914079	101.335404	35.175921	212.410145	100.56107

```
In [17]: # Calculate the percentage of customers who churned
churn_rate = df["churn"].mean() * 100
print("Churn rate:", churn_rate, "%")
Churn rate: 14.491449144914492 %
```

```
In [18]: # Select only numerical and boolean columns
numeric_data = df.select_dtypes(include=["number", "bool"])

# Calculate correlation with churn
churn_correlation = numeric_data.corr()["churn"]

# Display the correlations
churn_correlation
```

```
Out[18]: account length      0.016541
         area code          0.006174
         number vmail messages -0.089728
         total day minutes   0.205151
         total day calls     0.018459
         total day charge    0.205151
         total eve minutes   0.092796
         total eve calls     0.009233
         total eve charge    0.092786
         total night minutes 0.035493
         total night calls   0.006141
         total night charge  0.035496
         total intl minutes  0.068239
         total intl calls    -0.052844
         total intl charge   0.068259
         customer service calls 0.208750
         churn               1.000000
         Name: churn, dtype: float64
```

Business Insights and Conclusion

The exploratory data analysis was used to investigate patterns associated with customer churn.

The churn distribution showed that most customers remained with the company, while a smaller proportion of customers churned.

The boxplots, histograms, scatter plot and correlation analysis were used to investigate how customer characteristics and usage patterns relate to churn.

The analysis can help the company identify customers who may be at greater risk of leaving. The company could use these findings to improve customer retention strategies, such as providing targeted offers, reviewing pricing and improving customer service for customers showing signs of potential churn.